MINEARENA · PS-01 FINANCIAL FILING AUDITOR

AuditLoop

Project report: what we built, how it works, the problems we hit and how we fixed them

---

Team: Srujan Mirji (team lead; scoring, validation, verification, audit log, docs) · Zeeshan Madras (agent and backend) · Mayur Habib (SEC data and extraction) · Sai Sudheer Doddamani (frontend and UI/UX)

Repository: github.com/Srujanmirji/MINEARENA · **Built in:** 24 hours, 12–13 November 2026

Sources for this report: the git history (72 commits), commit messages, the project docs (PRD, architecture, API, data model, demo script, task files), and test and verification runs on 13 November 2026.

Contents

1. Executive summary
2. The problem and how we covered it
3. Architecture and tech stack
4. How it works, step by step
5. Team and contributions
6. How the 24 hours went
7. Challenges we faced and how we fixed them
8. Results and testing
9. Known limitations and next steps

Appendix: commands

1. Executive summary

AuditLoop reads a company's annual report (Form 10-K) from SEC EDGAR the way a human analyst would: from the HTML tables, never from the machine-readable answer key. It extracts 13 financial metrics, scores how confident it is in each one, checks that the numbers make sense together, approves the safe ones automatically and pauses for a human on anything risky. When the run finishes, it grades itself against the SEC's XBRL data and keeps a tamper-evident record of every decision.

Every requirement in sections 3.1 to 3.8 of the problem statement is built and was tested, along with all three bonus features (MCP tools, multi-period variance analysis with AI explanations, and adaptive thresholds).

Headline results (full run of all 5 demo companies, 13 November 2026)

Measure	Result
Extraction accuracy vs XBRL	56 / 58 = 96.6% (7 calculated values have no XBRL value to compare)
Auto-approved / escalated	36 / 29
Wrong auto-approvals	0 (both misses went to a human)
Confidence calibration	All 53 values scored above 0.85 were correct
Escalation precision	55.2% of escalations were real errors or anomalies
Reliability	0 crashes; repeated runs give identical results
Generalisation	9 companies never tuned on: 98 / 102 correct, 0 wrong auto-approvals
Tests	23 unit tests, 24/24 API end-to-end checks, module self-tests, UI build

2. The problem and how we covered it

The brief (MineArena PS-01) asks for an agent that fetches 10-K filings, extracts financial metrics with evidence, scores confidence, validates the numbers, pauses for a human when needed, resumes safely, verifies against XBRL and keeps an append-only audit log.

Requirement	How AuditLoop meets it	Where
3.1 SEC EDGAR integration	Real 10-Ks for AAPL, MSFT, NVDA, WMT, TSLA, plus any US company from a search box. Rate-limited, required User-Agent, disk cache so the demo runs offline.	edgar/
3.2 Extraction (≥ 8 metrics)	13 metrics per filing, each with period, unit, scale, source document, table, row label and the exact row text.	extract/
3.3 Confidence scoring	0.45 × extractor agreement + 0.30 × label match + 0.25 × period/unit. Threshold 0.85, configurable.	scoring/ confidence.py
3.4 Validation engine	A = L + E, segment sum, year-on-year (40%, configurable), plus scale, units, sign, margin, EPS sign, tie-outs, auditor's opinion, amended filing. Each check is passed, failed or N/A.	scoring/ validation.py
3.5 Human in the loop	Pauses on low confidence, weak label or any failed check. Shows value, score, source row and reason. Approve, correct or reject.	agent/, ui/
3.6 Persist and resume	LangGraph checkpoints in SQLite; decision saved first, then resume; every write keyed so nothing happens twice.	agent/graph.py, api/
3.7 Reviewer dashboard	React app: Runs, Review queue, Extracted values, Accuracy report, Audit log.	ui/frontend

Requirement	How AuditLoop meets it	Where
3.8 Verification and audit log	XBRL Company Facts matched by filing and period; accuracy report; append-only, hash-chained audit log.	verify/, audit/
Bonus: MCP tools	MCP server with filing, statement-table and XBRL tools; each workflow step may only use certain tools.	agent/mcp_tools.py
Bonus: variance analysis	Multi-year history, trend-aware year-on-year messages, AI explanation from the filing's MD&A.	extract/explain.py
Bonus: adaptive thresholds	Suggests a new confidence threshold from reviewer approvals and corrections, with a before/after impact table.	ui (Runs tab)

3. Architecture and tech stack

3.1 The big picture

The browser talks only to the FastAPI backend. The backend starts and resumes a LangGraph workflow. The workflow fetches from SEC EDGAR through a disk cache and writes to two SQLite databases.

Layer	What it does	Owner
React dashboard (ui/frontend)	Reviewer interface; calls the API only, through a typed client and a Vite proxy	Sai
FastAPI backend (api/main.py)	Starts runs, serves the review queue, records decisions, reports, audit log, config	Zeeshan
LangGraph workflow (agent/)	fetch → locate → extract → score → validate → route → auto-approve → human review ⇄ apply decision → verify → report	Zeeshan
SEC data (edgar/)	Ticker → CIK → latest 10-K; primary document; Company Facts; throttled, cached	Mayur
Extraction (extract/)	Strip inline XBRL, find statement tables, two independent extractors, tie-outs, auditor's opinion	Mayur
Scoring and validation (scoring/)	Confidence, validation checks, approve-or-escalate router	Srujan
Verification and audit (verify/, audit/)	XBRL comparison, accuracy report, hash-chained append-only log	Srujan

3.2 Tech stack

- **Python 3.11**, **LangGraph** (workflow, interrupt/resume, SQLite checkpointer), **FastAPI**, **SQLite**
- **pandas**, **BeautifulSoup / lxml** (HTML tables), **rapidfuzz** (row-label matching)
- **Groq** (openai/gpt-oss-120b) as the LLM, with **Gemini** as the backup provider; JSON mode, cached by prompt hash
- **Pydantic** (the ExtractedValue record and every API request body)
- **React + TypeScript + Vite + Tailwind** for the dashboard

3.3 Where the data lives

Location	Contents
cache/sec/	Every SEC response (submissions, 10-K HTML, Company Facts), keyed by URL hash

Location	Contents
cache/llm/	Every LLM answer, keyed by prompt hash, so runs are repeatable and work offline
data/checkpoints.db	The full workflow state after every step: what lets a paused review survive restarts
data/app.db	filings, extracted_values, review_decisions, audit_log, verification, run_reports, config

4. How it works, step by step

This is what happens when a reviewer clicks Run TSLA. The steps match section 4 of the brief.

1. **Filing retrieval.** The API turns the ticker into a run ID such as TSLA-2025 and starts the workflow in the background. fetch_filing looks up the CIK, finds the latest 10-K and records the accession number, fiscal year, period end and main document. A second click returns the same run instead of starting another.
2. **Cleaning and locating.** The 10-K is inline XBRL: its numbers are wrapped in tags that are effectively the answer key. We strip those tags but keep their text, so the system reads only what a human sees. The locator then finds the income statement, balance sheet, cash-flow and segment tables by their headings and their content, plus the scale ("in millions").
3. **Extraction.** Each table is read twice. The parser (pandas + rapidfuzz) matches row labels against known names. The LLM (Groq) must answer in JSON and quote the exact row; any answer whose quote is not found word for word in the table is rejected. If the parser found a clearly correct row and the LLM disagrees, the parser wins and the disagreement lowers confidence. Missing totals are calculated and marked as derived.
4. **Confidence scoring.** C = 0.45 × agreement + 0.30 × label + 0.25 × period/unit. Calculated values are capped at 0.75; a missing value scores 0.
5. **Validation.** A = L + E within 0.5%, segments sum to revenue, year-on-year change within 40%, scale sanity, units found, signs, operating margin, EPS sign, net-income and cash tie-outs, auditor's opinion and amended filing.
6. **Automated decision.** A value is auto-approved only if confidence ≥ 0.85, label match ≥ 0.80 and no check failed. Everything else is escalated with a plain-English reason; failed checks are queued first.
7. **Human review.** The workflow calls interrupt(), saves its state and stops. The dashboard shows the value, confidence breakdown, every check, the reason, and the original 10-K table with the row highlighted.
8. **Decision and resume.** The API saves the decision first (keyed by thread_id:value_id), then resumes from the checkpoint. A separate step writes the result exactly once, so a double click, retry or crash cannot record it twice.
9. **Verification and audit.** Only now does the system fetch XBRL Company Facts and compare, matching on filing number and period end so later restatements are ignored. The accuracy report is built, and every decision, human or automatic, is in the append-only, hash-chained audit log.

5. Team and contributions

We split the work by folder so four people could build in parallel without collisions (CONTRIBUTING.md). Each person owned their folders; changes to someone else's folder went through the lead and are marked "with lead approval" in the commit messages.

Member	Role and folders	Commits
Srujan Mirji	Team lead (P4). scoring/, verify/, audit/, docs/, data/mock_values.json. Also the pitch, checkpoints and integration fixes across the codebase.	41
Sai Sudheer Doddamani	Frontend and UI/UX (P3). ui/: landing page, dashboard design, motion and layout.	15
Zeeshan Madras	Agent and backend lead (P1). core/ (db, config), agent/, api/.	10
Mayur Habib	SEC data and extraction (P2). edgar/, extract/, cache/.	6

Commit counts measure how work was committed, not effort: Mayur's six commits delivered the whole SEC client and the first full extraction pipeline, and several of the lead's commits are fixes inside other members' folders.

5.1 Srujan Mirji: team lead

- **Planning:** wrote the PRD, architecture, API contract, data model, demo script and every member's task file; merged Zeeshan's plan (extract from the 10-K document, two extractors, the 3-part confidence formula).
- **Audit log:** append-only, SHA-256 hash-chained log with database triggers that block UPDATE and DELETE, a chain verifier and a live tamper test.
- **Scoring:** confidence score, validation engine and the approve-or-escalate router, including the label floor that stops a wrong row from scoring high.
- **Verification:** XBRL Company Facts comparison (accession + period matching), the accuracy report, and verify.run_all, which proves reliability with repeated full runs.
- **Integration fixes:** working LLM models and row rules (44/44), the API freeze under concurrent requests, false segment failures, scale and column-order bugs, statement location by content.
- **Features:** audit any US company from a search box, auditor's opinion check, tie-outs, sign-off and printable audit report, 4 extra metrics (13 per filing), the unit test suite, and demo_setup to build the judging state in one command.
- **Dashboard on the real API:** replaced hard-coded demo data with live API calls, removed the placeholder login and wallet screens, added the Extracted values tab.

5.2 Zeeshan Madras: agent and backend

- **Core contract:** shared models, SQLite database layer, the LangGraph graph skeleton and /health.
- **API v1:** runs, pending queue and decision endpoints, plus the five pause/resume tests that became the end-to-end test suite.
- **Hardening:** run list, values, validations, config, audit and source-evidence endpoints; report endpoints; API contract check.
- **Reliability:** failed runs report status=error with the step's message, even after a restart; fixed the empty duplicate run created by a second Run click.
- Committed the LLM cache at the 3 AM checkpoint so the demo runs offline.

5.3 Mayur Habib: SEC data and extraction

- **SEC client and cache:** ticker → CIK → latest 10-K, primary document, Company Facts; every request throttled, with the required User-Agent, cached to disk.

- **Extraction pipeline:** inline-XBRL cleaning, statement locator, the table parser (extractor A) and the LLM extractor (extractor B) with verbatim-snippet checking.
- **Bonus features:** B-2 AI variance explanations from the MD&A, and B-4 multi-year history for every metric.
- Multi-company parsing refinements so all five demo companies extract cleanly.

5.4 Sai Sudheer Doddamani: frontend and UI/UX

- **Review dashboard:** the first review queue, API client and full-screen dashboard with enter/exit transitions.
- **Landing page:** hero section, enterprise footer, scroll-driven reveals, parallax, progress bar, glass navbar, Lenis smooth scrolling.
- **Dashboard redesign:** dark glass sidebar, animated launch buttons, interactive stat tiles; fixed modal scrolling under Lenis.
- Layout fixes for compact viewports and visual polish against the design reference.

6. How the 24 hours went

Phase	What happened
1. Plan (first hours)	PRD, architecture, API, data model, demo script and task files written; folder ownership agreed; mock data with 8 real Apple values and 12 labelled test cases, so scoring could be built before extraction existed.
2. Build in parallel	Zeeshan: core contract, database, graph skeleton, API v1 with pause/resume tests. Srujan: audit log, scoring, validation, router, XBRL verification, accuracy report. Mayur: SEC client, cache, cleaning, locator, both extractors, bonus B-2 and B-4.
3. Integrate	The real pipeline replaced the mocks. First full run exposed retired LLM models and wrong-row picks; fixed to 44/44 on all five companies. verify.run_all added to prove it on every change.
4. Harden	API endpoints for the dashboard, error status after restart, duplicate-run fix, false segment failures, derived liabilities, LLM cache committed at 3 AM.
5. Dashboard	Sai's UI and review dashboard; Srujan wired every tab to the real API; API freeze under concurrent requests found and fixed (800/800 clean).
6. Generalise	Search any US company; tested on 9 companies never tuned on; fixed statement location, missing scale and column-order bugs; 0 wrong auto-approvals.
7. Go further	Auditor's opinion, tie-outs, amended-filing check, sign-off and printable audit report, MCP tool routing, 13 metrics, unit tests, adaptive threshold panel, pitch deck, variance analysis UI and tamper test.
8. Final checks (13 Nov)	Requirement-by-requirement audit, segment validation fixed for all five companies, full dry run, demo database built and servers verified.

7. Challenges we faced and how we fixed them

Every fix below was proved with a full graded run against XBRL before it was committed. The numbers come from the commit messages.

Challenge	What went wrong	How we fixed it	Proof
Retired LLM models	The planned Gemini and Llama models had been retired, and the LLM sometimes picked the wrong row (non-controlling interest, per-share or component lines).	Switched to Groq gpt-oss-120b with Gemini as automatic backup; shared row rules for both extractors; parser wins when it found a clearly correct row.	44/44 correct on all 5
Hard-coded period	period_start was hard-coded for Apple's 2024 year end.	One rule for every filing (one year earlier plus a day, leap-day safe); automated the Apple regression test.	Regression passes
False segment failures	The segment parser grabbed expense or gross-profit tables for MSFT, WMT and TSLA, raising a false hard failure on correct revenue.	First: reject implausible segment tables (show N/A). Final fix: a new parser for the three layouts real filings use, chosen by the table's own printed total.	Segment check passes for all 5
Hidden non-controlling interest	Derived total liabilities subtracted only the parent's equity, hiding NCI inside liabilities.	Derive from total equity including NCI, so A = L + E fails honestly with an NCI note.	WMT gap 2.2% explained
Empty duplicate run	A second Run click created a new run with 0 values whose decisions acted on the first run's rows.	Re-runs take over the filing's rows; a re-run is refused (409) while one is running or paused.	API test suite passes
API froze under load	About 8 overlapping requests froze every endpoint, even /health; only kill -9 recovered.	Checkpoint connection used under the saver's lock; one process-wide lock around each SQLite open/close.	800/800 requests clean, slowest 0.04 s
Wrong table for a new company	Vita Coco extracted nothing: the locator took a credit-loss note table after a heading mention.	A statement table must also contain its defining lines (revenue and net income, total assets, operating activities).	COCO 0 → 8/8
Missing scale	Pfizer and Home Depot auto-approved 6 values stored without their scale (164,683 instead of 164,683 million).	Recognise more scale wordings; new hard Units check: a dollar value with no scale always goes to a human.	Wrong auto-approvals 6 → 0
Years in reverse order	Alphabet and Amazon list the oldest year first, so current and prior were swapped.	Sort year columns latest first.	71 → 73/74; auto-approvals 46 → 61
Wrong row scored high	A wrong row such as "Total assets of discontinued operations" could still score about 0.87.	Use fuzz.ratio (not token_set_ratio) and a label floor of 0.80 that forces review.	Router tests
Answer key in the document	Inline XBRL tags in the 10-K would let extraction read the answer.	Strip ix tags before reading; XBRL is fetched only in verify/, enforced by MCP step permissions.	MCP blocks XBRL in extract
Restated numbers	Company Facts include restated values from later filings.	Match on accession number and period end.	Verify self-test
Repeated writes on resume	LangGraph re-runs the paused step from the top when it resumes.	The review step has no side effects; all writes happen in apply_decision, keyed by decision_id.	Duplicate decision is a no-op
Mock UI data	The first dashboard used hard-coded items, invented accuracy numbers and random audit hashes.	Every tab reads the real API; placeholder login and wallet removed; loading, error and empty states added.	UI build and type-check pass
Modal scrolling	Smooth-scroll library captured scroll events inside dashboard modals.	data-lenis-prevent on scrollable panels.	Fixed in UI

8. Results and testing

8.1 Accuracy by company (full run, 13 November 2026)

Company	Accuracy vs XBRL	Notes
Apple (AAPL)	100%	Segment sum passes (5 regions)
Microsoft (MSFT)	100%	3 segments
Walmart (WMT)	100%	A = L + E fails by 2.2%: non-controlling interest, explained to the reviewer
Nvidia (NVDA)	91.7%	Capex not found (went to a human); 9 values escalated for 53–98% growth
Tesla (TSLA)	91.7%	Long-term debt 6,736 vs XBRL 6,584 (went to a human); A = L + E off by 0.5%

8.2 Testing

- **Unit tests:** 23 tests for scoring, checks, extraction rules and the audit log.
- **Module self-tests:** every module has a __main__ check on mock or cached data (segments, confidence, validation, router, audit, verify).
- **API end-to-end (24/24):** pause on the first escalated item; second POST doesn't restart; 409/422 guards; approve resumes; kill and restart keeps the same paused item; duplicate decision is a no-op; row counts unchanged; report, CSV, audit chain, re-run ownership, crash status.
- **verify.run_all:** full runs of all five companies in a throwaway folder, checking crashes, identical results, wrong auto-approvals and the headline numbers.
- **Stress test:** 16 overlapping requests across 8 endpoints, 800/800 clean.
- **UI:** TypeScript type-check and production build.

9. Known limitations and next steps

Limitation	Why it matters	Next step
Escalation precision 55%	Some correct values still go to a human (e.g. calculated values, Nvidia's growth).	Use the adaptive threshold with more reviewer data; per-metric thresholds.
LLM output checked by hand	CLAUDE.md asks for Pydantic validation of LLM JSON; fields are validated manually today.	Add a Pydantic model in extract/llm_extractor.py.
Adaptive threshold is a rule	It moves the threshold ±0.02 from approval rates and lives in the dashboard.	Learn per-metric thresholds in the backend from the audit log.
Verification reads XBRL directly	The workflow fetches filings through MCP, but the verify step calls Company Facts directly.	Route verification through the get_xbrl_fact tool.
Landing page number	The landing page shows 97.8% (45/46) from an earlier run; the current full run is 96.6% (56/58).	Read the number from /report.
Single machine	SQLite and a local cache suit a demo, not many reviewers.	Postgres (LangGraph PostgresSaver), worker queue, SSO, secrets manager.

Uncommitted work at the time of this report

The final segment-validation fix (extract/segments.py, extract/locate.py, extract/extractor.py) and recent dashboard and scoring edits are in the working copy but not yet committed. extract/ is Mayur's folder, so he should review it before it is pushed.

Appendix: commands

Purpose	Command
Start the API	.venv/bin/python -m uvicorn api.main:app --port 8000
Start the dashboard	cd ui/frontend && npm run dev
Build the demo state	.venv/bin/python -m verify.demo_setup --reset (stop the API first)
Headline numbers	.venv/bin/python -m verify.run_all --runs 1
API end-to-end test	.venv/bin/python -m api.test_flow
Unit tests	.venv/bin/python -m unittest discover -s tests
MCP server	.venv/bin/python -m agent.mcp_tools --stdio